

Half-Moon Cookie: Private, Similarity-Based Blocklisting with TOCTOU-Attack Resilience

Xinyuan Zhang¹[0009-0003-8142-7785], Anrin Chakraborti²[0009-0006-2095-6992],
and Michael K. Reiter¹[0000-0001-7007-8274]

¹ Duke University, Durham, NC USA
{xinyuan.zhang887, michael.reiter}@duke.edu
² University of Illinois, Chicago, IL USA
anrin@uic.edu

Abstract. Blocklisting is a common technique for preventing the use of known malicious content. However, conventional blocklisting infrastructures require either the blocklist to be public or clients to reveal their queries to the blocklist server. We introduce a private blocklisting framework, *Half-Moon Cookie*, by which a client can check an item against a proprietary blocklist held by a server, to determine whether the item is close to any blocklist element in a metric space. Critically, our design separates the embedding step from the blocklist check, enabling independent choice of methods to compute them privately and efficiently. Still, this computation might be too costly to perform on the critical path of using the item, and so our design also supports a very efficient check that an item previously passed the blocklist check. In doing so, we support applications where one client can perform the blocklist check on the item before sending it, and recipients can more efficiently confirm the previous result before using the item, thereby avoiding TOCTOU attacks. We show how *Half-Moon Cookie* can be instantiated for similarity-based malware detection, to block malicious executables without revealing client inputs or disclosing the blocklist.

Keywords: Private blocklisting · TOCTOU attacks · Malware detection.

1 Introduction

Blocklisting is a core primitive of computer security that is relied upon daily to interfere with proliferation or use of malware [38, 50], unsolicited or fraudulent email [21], and other malicious artifacts [1, 5, 6, 19, 47]. When the blocklist and/or the artifacts checked against it are private or proprietary, verifying that an artifact is not on a blocklist might require using a cryptographic protocol to hide the blocklist from the client with the artifact and the artifact from the blocklist server. Performing such a blocklist check immediately prior to artifact use can be impractical in some cases, however, particularly if the blocklist check requires a rich cryptographic functionality (e.g., an *approximate* set intersection [10, 13, 16, 23, 24] using the client’s artifact and the server’s blocklist) that

incurs nontrivial cost. Checking the blocklist at an earlier, more manageable time can lead to time-of-check vs. time-of-use (TOCTOU) vulnerabilities [4], if the previous check is not reaffirmed right before the artifact’s use.

In this paper we propose a framework by which a client can perform a blocklist check—even an approximate one—at a server, while ensuring privacy of the artifact from the server and privacy of the blocklist from the client. If this *explicit check* passes, then the server adds a hiding and binding token for the artifact to an allowlist, which this or another client can later confirm has not been revoked, via an *implicit check* with the server. Like the explicit check, the implicit check preserves the privacy of the artifact (and the blocklist). However, it is much faster and so more practical to perform on the critical path of the artifact’s use, rendering TOCTOU attacks less of a threat. As such, a client can privately check an artifact against a server’s blocklist (using an explicit check) before sending it to many others, each of whom can quickly reaffirm this check (via an implicit check) right before use. Due to our design that allowlists an artifact after confirming it is not similar to anything on a blocklist, we refer to our design as “*Half-Moon Cookie*,” or “*Half Moon*,” for short.³

A core challenge in *approximate* blocklist checking is ensuring that the sender embeds its artifact correctly; otherwise, a malicious sender could check an embedding unrelated to the artifact it sends. A direct solution is to combine embedding and checking into a single immutable computation, e.g., via a monolithic garbled circuit [30]. However, as we show in Sec. 6, this approach does not scale with the number of blocklist elements (or clusters in the embedding space). Ideally, the computation should be independent of blocklist size. Even then, embedding large executables inside a garbled circuit remains prohibitively expensive.

Half Moon addresses all of the above challenges. We summarize our contributions as follows:

- We formally define the *Half Moon* primitive, and provide a general framework to design *Half Moon* protocols that support an *explicit check* and a more efficient *implicit check* that only takes 0.19 seconds and transmits 7.2 kB.
- We provide secure and efficient *Half Moon* implementations for blocklists that can be embedded into a metric space based on Hamming distance.
- We demonstrate how *Half Moon* can be applied to similarity-based malware detection in email attachments. For this application and representative parameters, we show that performing an explicit check using a monolithic garbled circuit is $231\times$ the cost of *Half Moon*’s method, while the network traffic is $43,505\times$ that of *Half Moon* for an average-size email attachment (194 kB).

2 Background: TOCTOU Attacks in Email Attachments and URLs

The class of vulnerabilities known as time-of-check vs. time-of-use (TOCTOU) arises when a resource deemed safe at validation time would be unsafe if rechecked

³ A half-moon, or black-and-white, cookie is a cookie with vanilla frosting on one half and chocolate frosting on the other [8].

at the moment of use [4]. TOCTOU vulnerabilities abound, and indeed the Amazon AWS outage in October 2025 has been at least partly attributed to one [9].

We consider policy enforcement on content (e.g., email attachments) received from an untrusted source, whether delivered directly or via reference (e.g., URLs). Empirical analyses from major security vendors have confirmed that TOCTOU-style email threats are widespread [7, 40]. In a representative TOCTOU exploitation scenario, a link initially deemed benign is later replaced with malicious content (e.g., ransomware) between inspection and user access, enabling credential theft and broader compromise.

A standard TOCTOU-resistant defense [11, 17, 39] requires the receiving client to query a remote server hosting a proprietary blocklist of malicious artifact descriptors (e.g., distance-preserving hashes [15, 34, 37, 42, 45]), immediately prior to the artifact’s execution. However, this model incurs privacy risks by exposing benign content descriptors, and additionally adds latency on the critical path. Our design improves this model in two ways. First, it enables privacy-preserving interaction, protecting both client content and the server’s blocklist. Second, it shifts the costly blocklist check to the sender, removing it from the receiver’s critical path.

Because the receiving client does not trust the sending client in our scenario, it requires evidence that the artifact was checked. A natural approach would be for the service to (blindly) sign the content after verifying that it is not blocklisted, and for the sender to forward this signature to the receiver. However, if the receiver only locally verifies this signature, then the TOCTOU vulnerability is reintroduced: the blocklist may have changed after the signature was issued. Content that was signed while policy-compliant may later become blocked. Therefore, the receiver must still interact with the service at the time of use to ensure that the artifact complies with the *latest* policy.

3 Definition

We are concerned with building a three-party primitive, *Half-Moon Cookie* (*Half Moon* in short), where a sending client (C_{snd}) checks whether its input w is present on a blocklist held by a server (S). If not, the server will add a token to an allowlist so that a receiving client (C_{rcv}) can confirm that the value w' it receives from C_{snd} is present on the allowlist. At the same time, w and w' are hidden from S , while S ’s blocklist is hidden from C_{snd} and C_{rcv} .

3.1 Notation

Let $g : \mathcal{K} \times \mathcal{D} \rightarrow \mathcal{R}$ denote a function family with *keyspace* $\mathcal{K}$, *domain* $\mathcal{D}$, and *range* $\mathcal{R}$. For $k \in \mathcal{K}$, we denote by $g_k : \mathcal{D} \rightarrow \mathcal{R}$ the function with $g_k(d) = g(k, d)$ for each $d \in \mathcal{D}$. Let $\mathbb{R}_{\geq 0}$ denote the non-negative real numbers. We use $\mathbb{F}$ to denote a finite field, and $p \in \mathbb{F}[x]$ to denote a polynomial whose coefficients are drawn from $\mathbb{F}$. A random polynomial $p(x) \xleftarrow{\$} \mathbb{F}[x]$ is a polynomial with coefficients uniformly sampled from $\mathbb{F}$. Given two vectors $\vec{v}_1, \vec{v}_2 \in \mathbb{F}^\theta$, $\vec{v}_1 \times \vec{v}_2 \in \mathbb{F}^\theta$, and “ $\times$ ” is the Hadamard product.

3.2 Half Moon Definition

Half Moon takes an input $w \in \mathcal{U}$ from $\mathcal{C}_{\text{snd}}$, where $\mathcal{U}$ is the input universe, and a list of items $\mathcal{U}_{\text{blk}} \subset \mathcal{U}$ from $\mathcal{S}$. The primitive should produce to $\mathcal{S}$ a binding and hiding mapping on w to add to $\mathcal{S}$'s allowlist *only if* $w \notin \mathcal{U}_{\text{blk}}$; if $w \in \mathcal{U}_{\text{blk}}$, the protocol should abort. $\mathcal{C}_{\text{snd}}$ then forwards w to $\mathcal{C}_{\text{rcv}}$. $\mathcal{C}_{\text{rcv}}$ will interact with $\mathcal{S}$ to privately test membership of w on $\mathcal{S}$'s allowlist.

That said, there are many scenarios where the blocklist $\mathcal{U}_{\text{blk}}$ is too large to be specified directly as an input to a *Half Moon* protocol (as we will show in Sec. 6). Therefore, we focus on settings where the set $\mathcal{U}_{\text{blk}}$ can be embedded into a metric space $\mathbb{F}^\theta$ with distance metric $d : \mathbb{F}^\theta \times \mathbb{F}^\theta \rightarrow \mathbb{R}_{\geq 0}$. The embedded set can be summarized by a much smaller set $\mathcal{L} \subset \mathbb{F}^\theta$. That is, in lieu of specifying $\mathcal{U}_{\text{blk}}$ to the protocol directly, $\mathcal{S}$ instead specifies a blocklist $\mathcal{L} \subset \mathbb{F}^\theta$, $|\mathcal{L}| \ll |\mathcal{U}_{\text{blk}}|$, and a threshold T such that elements of $\mathcal{U}_{\text{blk}}$, once embedded in $\mathbb{F}^\theta$, should fall within distance T of some element of $\mathcal{L}$. Mathematically, for an output $\vec{z}$ embedded from an element in $\mathcal{U}_{\text{blk}}$,

$$\text{blocked}^{\mathcal{L}, T}(\vec{z}) \stackrel{\text{def}}{=} \left(\exists \vec{l} \in \mathcal{L} : d(\vec{z}, \vec{l}) \leq T \right).$$

Naturally, such an embedding will not be perfect, and will introduce non-zero false-accept and false-reject rates. Here we consider a randomized embedding function family of the form $f : \mathcal{K}_f \times \mathcal{U} \rightarrow \mathbb{F}^\theta$. For instance, the keyspace $\mathcal{K}_f$ might include the random vectors used for projections in locality-sensitive hashing algorithms (e.g., [20]). We then define true- and false-reject rates as follows:

$$\begin{aligned} \text{TRR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}} &\stackrel{\text{def}}{=} \min_{w \in \mathcal{U}_{\text{blk}}} \mathbb{P} \left(\text{blocked}^{\mathcal{L}, T}(\vec{z}) \mid k_f \xleftarrow{\$} \mathcal{K}_f, \vec{z} \leftarrow f_{k_f}(w) \right) \\ \text{FRR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}} &\stackrel{\text{def}}{=} \max_{w \in \mathcal{U} \setminus \mathcal{U}_{\text{blk}}} \mathbb{P} \left(\text{blocked}^{\mathcal{L}, T}(\vec{z}) \mid k_f \xleftarrow{\$} \mathcal{K}_f, \vec{z} \leftarrow f_{k_f}(w) \right) \end{aligned}$$

In words, $\text{TRR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}}$ is a lower bound on the probability of blocking an input w that should be blocked, and $\text{FRR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}}$ is an upper bound on the probability of blocking an input w that should not be blocked. As usual, false- and true-accept rates can be defined by $\text{FAR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}} \stackrel{\text{def}}{=} 1 - \text{TRR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}}$ and $\text{TAR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}} \stackrel{\text{def}}{=} 1 - \text{FRR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}}$, respectively. Naturally, we strive to maximize $\text{TRR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}}$ and $\text{TAR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}}$. The definition implies that inputs are not uniformly distributed, while keys are. This corresponds to the applications, where inputs are user-chosen.

Definition 1 (Half-Moon Cookie) A *Half-Moon Cookie* for blocklist $\mathcal{U}_{\text{blk}}$ is a three-party protocol among a sending client ($\mathcal{C}_{\text{snd}}$), a receiving client ($\mathcal{C}_{\text{rcv}}$), and a server ($\mathcal{S}$), consisting of two phases: *explicit check* and *implicit check*.

- **Explicit Check:** $\mathcal{C}_{\text{snd}}$ inputs $w \in \mathcal{U}$ and $\mathcal{S}$ inputs $\mathcal{U}_{\text{blk}} \subset \mathcal{U}$, $\mathcal{L} \subset \mathbb{F}^\theta$, $T \in \mathbb{R}_{\geq 0}$ (with both $\mathcal{L}$ and T computed from $\mathcal{U}_{\text{blk}}$). This phase succeeds with probability at least $\text{TAR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}}$ if $w \notin \mathcal{U}_{\text{blk}}$ and with probability at most $\text{FAR}_{\mathcal{L}, T}^{\mathcal{U}_{\text{blk}}}$ otherwise. If this phase succeeds, then $\mathcal{C}_{\text{snd}}$ outputs 1 and $\mathcal{S}$ outputs state ϕ . Otherwise, $\mathcal{S}$ aborts and $\mathcal{C}_{\text{snd}}$ outputs 0.

- **Implicit Check:** A client $C \in \{C_{\text{snd}}, C_{\text{rcv}}\}$ inputs $w' \in \mathcal{U}$ and the server S inputs ϕ . This phase succeeds if ϕ contains an output from a successful explicit check using w' . If so, C outputs 1. Otherwise, C outputs 0. S has no output.

We will define the guarantees that we want to provide from these steps depending on the following threat models.

3.3 Threat model

The threat models we consider for our applications in Sec. 5 are:

- T1** The server S and receiving client C_{rcv} are honest, but C_{snd} is malicious, i.e., C_{snd} deviates from the protocol to circumvent the check. More specifically, a malicious C_{snd} should be prevented from causing S to store a token for a blocklisted input during explicit check, which is successfully verified during an honest implicit check initiated by C_{rcv} . The malicious C_{snd} may also invoke an implicit check to learn more information about the allowlist token, e.g., the secrets contributed by S in its computation. We require that the allowlist token is binding, meaning that C_{snd} cannot generate an alternative input w' that is consistent with an existing allowlist token. In addition, the protocol should not leak more information about $\mathcal{U}_{\text{blk}}$ to C_{snd} than the decision ($w \in \mathcal{U}_{\text{blk}}$ or not) implies. We note that S needs to limit the number of explicit check attempts. Otherwise, C_{snd} can online guess the blocklist entries.
- T2** C_{snd} and C_{rcv} are honest, but S is honest-but-curious. In this case, our concern is leaking information about w to S from its interaction with C_{snd} and C_{rcv} . That is, the allowlist token is hiding, and S 's ability to distinguish between different values of w is negligible.

We do not consider a threat model in which S is malicious, simply because we do not find that reasonable in our applications, where S is contracted to, say, provide malware defense for C_{snd} and C_{rcv} . In such cases, S is incentivized to perform policy checking correctly. In addition, we assume that C_{rcv} is honest since it is incentivized to perform the check correctly.

4 Half Moon Framework

4.1 Ideal Functionality

Half Moon is realized with a general cryptographic framework. In this section, we will describe the individual building blocks of this framework as ideal functionalities, and then show how to combine them. In Sec. 5, we will describe how to realize these ideal functionalities.

Recall that *Half Moon* has two phases that share states, an explicit check and an implicit check. To accommodate both threat models **T1** and **T2**, we will further split the explicit check functionality into two separate ideal functionalities. Specifically, the explicit check can be viewed as a two-step process: i) *Embed-and-Map* $\mathcal{F}_{\text{EM}}$, that embeds C_{snd} 's input into the metric space $\mathbb{F}^\theta$, and ii) *Test-and-Commit* $\mathcal{F}_{\text{TC}}$, that performs the predicate check $\text{blocked}^{\mathcal{L}, T}(\cdot)$ on the

Parameters: An embedding function $f : \mathcal{K}_f \times \mathcal{U} \rightarrow \mathbb{F}^\theta$; a non-programmable random oracle $H : \{0, 1\}^* \rightarrow \mathbb{F}^\theta$; and a one-time pairwise unpredictable permutation family over the metric space $P : \mathcal{K}_P^\theta \times \mathbb{F}^\theta \rightarrow \mathbb{F}^\theta$.

$\mathcal{F}_{EM}$: Embed-and-Map functionality

Inputs: C_{snd} inputs $w \in \mathcal{U}$ and $\vec{m} \in \mathbb{F}^\theta$. S inputs $k_f \in \mathcal{K}_f$, $\vec{s} \in \mathbb{F}^\theta$, and $\vec{k1}_P, \vec{k2}_P \in \mathcal{K}_P^\theta$.

Output: $\mathcal{F}_{EM}$ outputs $\vec{p1} \leftarrow P_{\vec{k1}_P}(f_{k_f}(w))$ and $\vec{p2} \leftarrow P_{\vec{k2}_P}(\vec{y})$ where $\vec{y} \leftarrow \vec{s} \times H(w, f_{k_f}(w), k_f, \vec{m})$ to C_{snd} ; and S has no output.

$\mathcal{F}_{TC}$: Test-and-Commit Functionality

Inputs: C_{snd} inputs $\vec{p1}, \vec{p2} \in \mathbb{F}^\theta$. S inputs $\mathcal{L} \subset \mathbb{F}^\theta$, $T \in \mathbb{R}_{\geq 0}$, and keys $\vec{k1}_P, \vec{k2}_P \in \mathcal{K}_P^\theta$.

Output: If $\neg \text{blocked}^{\mathcal{L}, T}(P_{\vec{k1}_P}^{-1}(\vec{p1}))$, then $\mathcal{F}_{TC}$ outputs 1 and *nonce* to C_{snd} and $\vec{\gamma} \leftarrow P_{\vec{k1}_P}^{-1}(\vec{p1}) + P_{\vec{k2}_P}^{-1}(\vec{p2})$ to S . Otherwise, $\mathcal{F}_{TC}$ outputs 0 to C_{snd} ; S aborts and learns $\{\vec{l} \in \mathcal{L} : d(P_{\vec{k1}_P}^{-1}(\vec{p1}), \vec{l}) \leq T\}$.⁴

$\mathcal{F}_{IC}$: Implicit Check Functionality

Inputs: C_{rcv} inputs $\vec{z1}, \vec{z2} \in \mathbb{F}^\theta$. S inputs $\vec{s}, \vec{\gamma} \in \mathbb{F}^\theta$.

Output: $\mathcal{F}_{IC}$ outputs $\vec{\gamma}' \leftarrow \vec{z1} + \vec{s} \times \vec{z2}$ to S . If $\vec{\gamma} \stackrel{?}{=} \vec{\gamma}'$, C_{rcv} receives 1; otherwise, C_{rcv} receives 0.

Fig. 1: Ideal Functionalities used in *Half Moon*

embedded input against the embedded blocklist $\mathcal{L}$ provided by S . Lastly, during implicit check, C_{rcv} and S participate in iii) *Implicit check* $\mathcal{F}_{IC}$, that allows fast verification on a previously checked input. We present the details for the three ideal functionalities, which are summarized in Fig. 1.

4.2 Embed-and-Map

Given C_{snd} 's input w and a random mask $\vec{m}$, and S 's keys $k_f, \vec{k1}_P, \vec{k2}_P$, and a random mask $\vec{s}$, this functionality first computes $\vec{p1} \leftarrow P_{\vec{k1}_P}(f_{k_f}(w))$ and $\vec{p2} \leftarrow P_{\vec{k2}_P}(\vec{s} \times H(w, f_{k_f}(w), k_f, \vec{m}))$ where f is an embedding function, $P : \mathcal{K}_P^\theta \times \mathbb{F}^\theta \rightarrow \mathbb{F}^\theta$ is a one-time pairwise unpredictable permutation family (defined in App. A), and H is a hash function modeled as a non-programmable random oracle (defined in the full version [54]). $\vec{p1}$ and $\vec{p2}$ are expected to be inputs to the next functionality (i.e., Test-and-Commit). $\vec{p1}$ will be used (after inversion) for testing against the blocklist for approximate matching, and $\vec{p2}$ will be used

⁴ Note that the server can implicitly learn this information even if the functionality did not return it, as an abort would imply w is close to one of the server's inputs.

for generating a hiding and binding token as an allowlist entry for the implicit check.

4.3 Test-and-Commit

During Test-and-Commit, the functionality $\mathcal{F}_{\text{TC}}$ inverts $\vec{p}_1$ and checks whether $P_{k1p}^{-1}(\vec{p}_1)$ (which should be equal to $f_{k_f}(w)$ if C_{snd} is honest) is blocklisted. If not, $\mathcal{F}_{\text{TC}}$ inverts $\vec{p}_2$ and outputs an allowlist token $\vec{\gamma} \leftarrow P_{k1p}^{-1}(\vec{p}_1) + P_{k2p}^{-1}(\vec{p}_2)$ to S for updating the allowlist. If C_{snd} inputs the same $\vec{p}_1$ and $\vec{p}_2$ that it received from $\mathcal{F}_{\text{EM}}$ to $\mathcal{F}_{\text{TC}}$, then $\vec{\gamma} = f_{k_f}(w) + \vec{s} \times H(w, f_{k_f}(w), k_f, \vec{m})$.

Subsequently, C_{snd} forwards w and $\vec{m}$ to C_{rcv} to use in an implicit check, while S stores the key k_f , the mask $\vec{s}$, and the explicit check output $\vec{\gamma}$ for the allowlist. For a successful check, S discards $k1p, k2p$ stored from $\mathcal{F}_{\text{EM}}$, and adds the *token* $\langle \text{nonce}, k_f, \vec{s}, \vec{\gamma} \rangle$ to its allowlist. We will explain the use of *nonce* when describing $\mathcal{F}_{\text{IC}}$. C_{snd} only learns whether the explicit check succeeded in setting up an allowlist token, *but does not learn the token itself*.

4.4 Implicit Check

During implicit check, C_{rcv} can be provided the embedding key k_f by S in the clear, because each explicit check requires a new embedding key, and once C_{snd} has set up $\vec{\gamma}$ during the explicit check, k_f does not provide any additional advantage to a malicious C_{snd} in evading the policy check. C_{rcv} uses the forwarded inputs $w, \vec{m}$ from C_{snd} to re-compute $\vec{z}_1 \leftarrow f_{k_f}(w)$ and $\vec{z}_2 \leftarrow H(w, f_{k_f}(w), k_f, \vec{m})$ locally. Then S and C_{rcv} interactively and privately evaluate $\vec{\gamma}' \leftarrow \vec{z}_1 + \vec{s} \times \vec{z}_2$. Only S receives the output $\vec{\gamma}'$ and no further information is given to either C_{rcv} or S . S compares $\vec{\gamma}'$ against the $\vec{\gamma}$ stored from a previous explicit check. C_{rcv} learns whether the implicit check has succeeded, implying that $\vec{\gamma}'$ has matched an $\vec{\gamma}$ from some allowlist token $\langle \text{nonce}, k_f, \vec{s}, \vec{\gamma} \rangle$ stored at S . As we will show, $\vec{\gamma}$ is computationally binding, and so it is infeasible for a malicious C_{snd} to find some pair of inputs $w', \vec{m}'$ that match an allowlist token generated from different inputs $w, \vec{m}$ during an explicit check.

Allowlist: Each explicit check inserts a new entry into the allowlist. The implicit check is indexed by a nonce *nonce* and implemented as a fast hashtable lookup. *nonce* is derived from the explicit check timestamp and is independent of the file content, preventing leakage about w . S 's storage overhead grows linearly with the number of explicit checks. This overhead is inherent to our security model: each entry must include a per-check S secret to prevent C_{snd} probing.

As discussed in Sec. 2, *Half Moon* enables TOCTOU resistance. In particular, S can revoke allowlist entries when the blocklist is updated. C_{rcv} must then run the explicit check, ensuring checks are performed against the latest policy.

Putting it all together: We present the full framework in Fig. 2, assuming that $\mathcal{F}_{\text{EM}}$, $\mathcal{F}_{\text{TC}}$, and $\mathcal{F}_{\text{IC}}$ are secure under the threat models in Sec. 3.3. We also provide a proof sketch here; a formal security proof, based on the properties of P and H , appears in the full version [54].

Half Moon Framework

Inputs: C_{snd} inputs $w \in \mathcal{U}$. S inputs $\mathcal{L} \subset \mathbb{F}^\theta$, $T \in \mathbb{R}_{\geq 0}$, and $\vec{k1_P}, \vec{k2_P} \in \mathcal{K}_P^\theta$.

Explicit check:

1. C_{snd} selects $\vec{m} \xleftarrow{\$} \mathbb{F}^\theta$. S selects $\vec{s} \xleftarrow{\$} \mathbb{F}^\theta$ and $k_f \xleftarrow{\$} \mathcal{K}_f$.
2. C_{snd} and S call $\mathcal{F}_{\text{EM}}$. C_{snd} inputs w and $\vec{m}$. S inputs $k_f, \vec{s}, \vec{k1_P}, \vec{k2_P}$. C_{snd} learns $\vec{p1} \leftarrow P_{\vec{k1_P}}^{-1}(f_{k_f}(w))$ and $\vec{p2} \leftarrow P_{\vec{k2_P}}^{-1}(\vec{y})$ where $\vec{y} \leftarrow \vec{s} \times H(w, f_{k_f}(w), k_f, \vec{m})$. S gets no output.
3. C_{snd} sets $\vec{p1}' \leftarrow \vec{p1}$, and $\vec{p2}' \leftarrow \vec{p2}$.
4. C_{snd} and S call $\mathcal{F}_{\text{TC}}$. C_{snd} inputs $\vec{p1}'$ and $\vec{p2}'$. S inputs $\mathcal{L}, T, \vec{k1_P}$, and $\vec{k2_P}$. If $\neg \text{blocked}^{\mathcal{L}, T}(P_{\vec{k1_P}}^{-1}(\vec{p1}'))$ then C_{snd} receives 1 and *nonce*; S receives $\vec{\gamma} \leftarrow P_{\vec{k1_P}}^{-1}(\vec{p1}') + P_{\vec{k2_P}}^{-1}(\vec{p2}')$. Otherwise, C_{snd} receives 0; S aborts and learns $\{\vec{l} \in \mathcal{L} : d(P_{\vec{k1_P}}^{-1}(\vec{p1}'), \vec{l}) \leq T\}$.

Implicit check:

5. C_{snd} sets *nonce'* $\leftarrow$ *nonce*, $w' \leftarrow w$, $\vec{m}' \leftarrow \vec{m}$ and sends *nonce'*, w' and $\vec{m}'$ to C_{rcv} .
6. C_{rcv} sends *nonce'* to S . S indexes the allowlist ϕ with *nonce'* and obtains $\langle k_f, \vec{s}, \vec{\gamma} \rangle$. S sets $k'_f \leftarrow k_f$, $\vec{s}' \leftarrow \vec{s}$, and $\vec{\gamma}' \leftarrow \vec{\gamma}$. S sends k'_f to C_{rcv} .
7. C_{rcv} computes $\vec{z1} \leftarrow f_{k'_f}(w')$ and $\vec{z2} \leftarrow H(w', f_{k'_f}(w'), k'_f, \vec{m}')$.
8. C_{rcv} and S call $\mathcal{F}_{\text{IC}}$. C_{rcv} inputs $\vec{z1}, \vec{z2}$, and S inputs $\vec{s}'$. S receives $\vec{\gamma}^\dagger \leftarrow \vec{z1} + \vec{s}' \times \vec{z2}$.
9. S checks whether $\vec{\gamma}^\dagger \stackrel{?}{=} \vec{\gamma}'$. If so, C_{rcv} receives 1; otherwise, C_{rcv} receives 0.

Fig. 2: *Half Moon* framework

Theorem 1. Assuming that there are secure protocols realizing $\mathcal{F}_{\text{EM}}$, $\mathcal{F}_{\text{TC}}$ and $\mathcal{F}_{\text{IC}}$, the framework described in Fig. 2 securely realizes *Half Moon* against the adversaries described in Sec. 3.3.

Proof (sketch): We prove that the *Half Moon* framework is secure against threat models **T1** and **T2**.

Threat model T1: Consider an adversary that interacts with S . During explicit check, the adversary may try to produce a file input w that should be blocked, but is not detected as such due to a false negative under the embedding function f . Since the embedding key k_f is randomly selected and unknown to the adversary, it is hard for the adversary to produce such a false negative.

Alternatively, the adversary may produce file inputs $w \neq w'$ and masks $\vec{m} \neq \vec{m}'$, such that the explicit check succeeds with $w, \vec{m}$ as inputs. However, the adversary sends $w', \vec{m}'$ to an honest receiving client C_{rcv} and the implicit check with $w', \vec{m}'$ succeeds. This implies that the adversary tricked S into authenticating an input for an honest C_{rcv} that was not verified by a previous

explicit check. If $\vec{p_1'} \neq \vec{p_1}$ or $\vec{p_2'} \neq \vec{p_2}$, then $\vec{\gamma}$ is unpredictable without $\vec{k_{1P}}, \vec{k_{2P}}$ (Definition 4). In that case, the adversary produces $w', \vec{m}'$ that evaluates to $\vec{\gamma}$ only with probability $\leq |\mathbb{F}|/|\mathcal{K}_P|$. So, it must be that $f_{k_f}(w') = f_{k_f}(w)$ and $H(w', f_{k_f}(w'), k_f, \vec{m}') = H(w, f_{k_f}(w), k_f, \vec{m})$. Thus, an adversary that produces $w \neq w'$ such that $H(w', f_{k_f}(w'), k_f, \vec{m}') = H(w, f_{k_f}(w), k_f, \vec{m})$, also finds a collision in the random oracle H , which occurs with probability negligible in $\log |\mathbb{F}|$.

Threat model T2: The only information S obtains is whether $\text{blocked}_{\mathcal{U}_{\text{bik}}}^{\mathcal{L},T}(f_{k_f}(w))$ and the value $\vec{\gamma}$. As mentioned in Fig. 1, when $\text{blocked}_{\mathcal{U}_{\text{bik}}}^{\mathcal{L},T}(f_{k_f}(w))$, there would be inevitable leakage to S. So we ensure that S learns strictly no information about w if $\neg \text{blocked}_{\mathcal{U}_{\text{bik}}}^{\mathcal{L},T}(f_{k_f}(w))$. Since $\vec{m}$ is selected randomly and H is a non-programmable random oracle, $H(w, f_{k_f}(w), k_f, \vec{m})$ is uniformly distributed $\mathbb{F}^\theta$. Thus, $\vec{\gamma}$ hides w from S. $\square$

5 Protocol

In this section, we describe how to efficiently realize the ideal functionalities leveraged in *Half Moon*. Fig. 3a depicts the explicit check and Fig. 3b visualizes the implicit check.

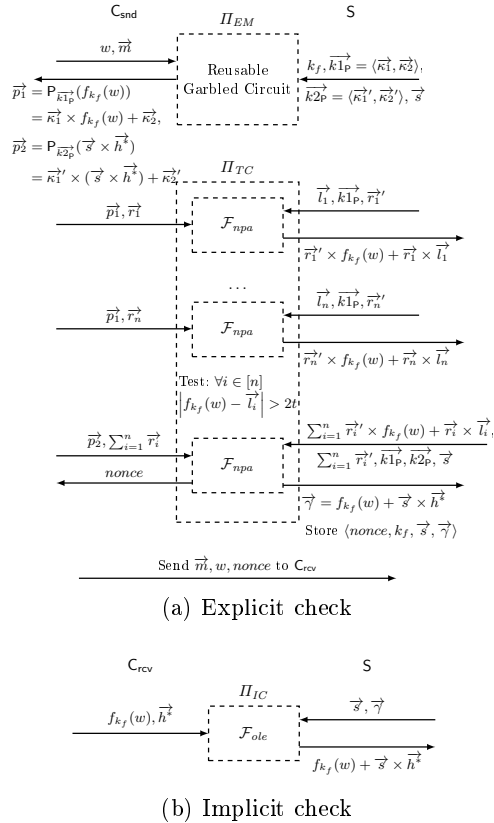


Fig. 3: High-level description of *Half Moon* instantiation ($\vec{h}^* \leftarrow \mathbf{H}(w, f_{k_f}(w), k_f, \vec{m})$)

or both of the parties, where “ $\times$ ” is the Hadamard product. The functionality does not reveal $\vec{r}_1, \vec{z}$ to the server and $\vec{r}_2, \vec{z}'$ to the client (beyond what can be learned from the output).

5.2 Protocol for Embed-and-Map

Malware Hashing and Embedding Function In $\mathcal{F}_{\text{EM}}$, f is a function mapping w to a vector in $\mathbb{F}^\theta$, that generates close outputs for similar inputs. We use a hashing scheme embedding files into Hamming space, $\text{LSH} : \mathcal{K}_f \times \{0, 1\}^* \rightarrow \{0, 1\}^\delta$ (see Sec. 6), and define an injective map $M_{\mathcal{X}}$ from bit-vectors to $\mathbb{F}^\theta$.

Definition 2 (Hamming to $\mathbb{F}^\theta$ map) *The mapping function $M_{\mathcal{X}} : \{0, 1\}^\delta \rightarrow \mathbb{F}^\theta$ is parameterized by the set of arbitrary points $\mathcal{X} \leftarrow \{x_1, \dots, x_\theta\} \subset \mathbb{F}$ for $\theta \geq 2\delta + 1$, and works as follows:*

1. **Map to set:** *Given $\vec{w} \in \{0, 1\}^\delta$, use an injective function $m : \{0, 1\} \times [\delta] \rightarrow \mathbb{F}$ to create a set $\mathcal{S}_{\vec{w}} = \{m(\vec{w}[j], j)\}_{j \in [\delta]}$, where $[\delta] = \{1, \dots, \delta\}$ and $\vec{w}[j]$ is the j -th component of $\vec{w}$.*
2. **Create a polynomial:** *Compute a polynomial $e_{\vec{w}}(x) \leftarrow \prod_{s \in \mathcal{S}_{\vec{w}}} (x - s)$.*
3. **Evaluate the polynomial:** *Evaluate $e_{\vec{w}}(x)$ at the points in $\mathcal{X}$ and output a vector $\vec{v} \leftarrow \langle e_{\vec{w}}(x_1), \dots, e_{\vec{w}}(x_\theta) \rangle$.*

Embedding with Reusable Garbled Circuits Given the description of the embedding function f , a protocol realizing $\mathcal{F}_{\text{EM}}$ will interactively compute $\vec{p}_1 \leftarrow \text{P}_{\langle \vec{\kappa}_1, \vec{\kappa}_2 \rangle}(f_{k_f}(w)) = \vec{\kappa}_1 \times f_{k_f}(w) + \vec{\kappa}_2$, and $\vec{p}_2 \leftarrow \text{P}_{\langle \vec{\kappa}_1', \vec{\kappa}_2' \rangle}(\vec{y}) = \vec{\kappa}_1' \times \vec{y} + \vec{\kappa}_2'$, where $\vec{y} \leftarrow \vec{s} \times \text{H}(w, f_{k_f}(w), k_f, \vec{m})$ and $f_{k_f}(w) = M_{\mathcal{X}}(\text{LSH}_{k_f}(w))$. While doing so, it hides S 's inputs (i.e., $k_f, \vec{k1_P}, \vec{k2_P}, \vec{s}$) from C_{snd} , where $\vec{k1_P} = \langle \vec{\kappa}_1, \vec{\kappa}_2 \rangle$ and $\vec{k2_P} = \langle \vec{\kappa}_1', \vec{\kappa}_2' \rangle$. It also hides C_{snd} 's inputs (i.e., $w, \vec{m}$) from S . We realize this requirement using garbled circuits [52, 53]. A naive approach is impractical, however, as the circuit size scales linearly with the input file w .

To address this, we employ reusable garbled circuits. Unlike classical Yao garbling, which requires fresh circuits per execution, reusable garbled circuits support repeated evaluation with different inputs. We adopt optimizations from the CRGC framework [28], which transforms a Boolean circuit C and garbler input a into obfuscated counterparts C' and a' such that $C(a, b) = C'(a', b)$ for any evaluator input b . This allows the evaluator to treat the garbled circuit as a reusable “black-box program” without learning intermediate state across executions. Our construction relies on two key CRGC security guarantees:

1. Balanced gates (e.g., XOR/XNOR) in the reusable portion hide garbler input bits under CRGC security assumptions [28, Lemma 2].
2. Gates in the non-reusable portion do not leak garbler input bits under standard Yao garbling security [28, Lemma 4].

5.3 Protocol for Test-and-Commit

Test: To begin with, the functionality $\mathcal{F}_{\text{TC}}$ privately compares $\mathcal{C}_{\text{snd}}$'s input $f_{k_f}(w) \leftarrow \mathcal{P}_{k1p}^{-1}(\vec{p}_1)$ against $\mathcal{S}$'s blacklist embeddings. To perform this comparison privately, we can apply an *assumption-free*⁵ fuzzy PSI scheme that compares vectors in Hamming space. However, $f_{k_f}(w)$ is not a bit vector, but rather a vector in a field $\mathbb{F}^\theta$ generated using the map $M_{\mathcal{X}}$ (Definition 2). We overcome this problem by defining a metric such that computing the distance between two vectors in $\mathbb{F}^\theta$ provides the Hamming distance between their preimages under $M_{\mathcal{X}}$. Then, we modify an existing assumption-free fuzzy PSI [13] protocol to compute Hamming distance using this metric.

This metric is called *symmetric uncommon roots* (SUR), defined as follows. For polynomial $p(x) \in \mathbb{F}[x]$, let $\mathcal{R}_{p(x)}$ be its set of (unique) roots. Let $\mathcal{S}_1 \triangle \mathcal{S}_2$ denote the symmetric difference between sets $\mathcal{S}_1$ and $\mathcal{S}_2$, i.e., $(\mathcal{S}_1 \setminus \mathcal{S}_2) \cup (\mathcal{S}_2 \setminus \mathcal{S}_1)$.

Definition 3 (Symmetric Uncommon Roots $\|\cdot\|_{\text{SUR}}$) Let $\mathcal{X} \leftarrow \{x_1, \dots, x_{2\delta+1}\}$ be a set of $2\delta + 1$ unique, fixed elements in $\mathbb{F}$. For vector $\vec{v} = \langle v_1, \dots, v_{2\delta+1} \rangle \in \mathbb{F}^{2\delta+1}$, let $p_{\vec{v}}(x) \in \mathbb{F}[x]$ be the unique polynomial of degree $\leq 2\delta$ that interpolates the set of points $\{(x_k, v_k)\}_{k \in [2\delta+1]}$. Then, for $\vec{v}_1, \vec{v}_2 \in \mathbb{F}^{2\delta+1}$,

$$\begin{aligned} \|\vec{v}_1 - \vec{v}_2\|_{\text{SUR}} &\stackrel{\text{def}}{=} \left| \mathcal{R}_{p_{\vec{v}_1}(x)} \triangle \mathcal{R}_{p_{\vec{v}_2}(x)} \right| \\ &= \left| \mathcal{R}_{p_{\vec{v}_1}(x)} \right| + \left| \mathcal{R}_{p_{\vec{v}_2}(x)} \right| - 2 \cdot \left| \mathcal{R}_{p(x)} \right| \end{aligned}$$

where $p(x) = \gcd(p_{\vec{v}_1}(x), p_{\vec{v}_2}(x))$ is the greatest common divisor of $p_{\vec{v}_1}(x)$ and $p_{\vec{v}_2}(x)$.

Since the cardinality of symmetric difference between two sets is a metric, SUR is also a metric. Moreover, the Hamming distance between two bit vectors $\vec{w} \in \{0, 1\}^\delta$ and $\vec{l} \in \{0, 1\}^\delta$ can be computed by

$$\left\| \vec{w} - \vec{l} \right\|_H \leq t \iff \left\| M_{\mathcal{X}}(\vec{w}) - M_{\mathcal{X}}(\vec{l}) \right\|_{\text{SUR}} \leq T \quad (1)$$

where $T = 2t$.

One advantage of using $\|\cdot\|_{\text{SUR}}$ is that we can compute the distance between two vectors contributed by mutually untrusting parties *without explicitly revealing the vectors to each other*. The process is as follows. $\mathcal{C}_{\text{snd}}$ and $\mathcal{S}$ sample random polynomials $r_1(x) \xleftarrow{\$} \mathbb{F}[x]$ and $r_2(x) \xleftarrow{\$} \mathbb{F}[x]$ respectively, of degree δ . Then, they locally compute $\vec{r}_1 \leftarrow \langle r_1(x) \rangle_{x \in \mathcal{X}}$ and $\vec{r}_2 \leftarrow \langle r_2(x) \rangle_{x \in \mathcal{X}}$. For each element l in $\mathcal{S}$'s blacklist, the protocol uses calls to $\mathcal{F}_{\text{npa}}$ to compute $\vec{r}_1 \times f_{k_f}(w) + \vec{r}_2 \times f_{k_f}(l)$, which preserves the SUR distance with high probability, enabling private distance evaluation (see the full version [54]).

⁵ An assumption-free fuzzy PSI scheme does not require any specific conditions on the data distribution. Since the output of malware hashes are not constrained to any specific distribution, we must leverage assumption-free schemes only.

Commit: If the distance check is satisfied, Π_{TC} computes $\vec{\gamma} \leftarrow P_{k1p}^{-1}(\vec{p}_1) + P_{k2p}^{-1}(\vec{p}_2)$ via $\mathcal{F}_{npa}$, yielding an authentication token while hiding all intermediate values. Finally, S stores the output authentication token $\langle nonce, k_f, \vec{s}, \vec{\gamma} \rangle$, while C_{snd} receives no output from Π_{TC} other than a confirmation that the check passes. C_{snd} can now forward $\vec{m}, w, nonce$ to any C_{rcv} .

5.4 Implicit Check

After receiving the file w , the mask $\vec{m}$ and a nonce $nonce$, the client C_{rcv} forwards $nonce$ to S . $nonce$ allows the server to identify the authentication token $\vec{\gamma}$ for the file, the embedding key k_f , and the mask $\vec{s}$. C_{rcv} recomputes $f_{k_f}(w)$ and $H(w, f_{k_f}(w), k_f, \vec{m})$ after obtaining k_f from S . Then, C_{rcv} and S invoke $\mathcal{F}_{ole}$, to compute $\vec{\gamma}$, which is output to S (see Fig. 3b). Finally, C_{rcv} learns whether $\vec{\gamma}$ was previously stored at S , which implies w was checked. The full protocol is described in the full version [54].

6 Evaluation

We implemented the *Half Moon*-based malware detection in C++11⁶. We benchmarked these implementations on a Ubuntu 22.04 machine with 12 cores and 32GB of RAM as the server, and multiple machines that are each equipped with 4 cores and 16GB of RAM as the clients. We used the emp-toolkit [2] for OT and garbled circuit operations, and CRGC for reusable garbled circuits. We used the NTL [3] library for finite field arithmetic. The operations were performed in a 128-bit prime-order field. We implemented Ghosh et al.’s OLE [27] and Ghosh et al.’s PSI [25] as building blocks for *Half Moon*. Our construction works for any metrics that can be embedded into Hamming distance.

6.1 Dataset

As discussed in Sec. 2, our *Half Moon*-based malware detection can be applied to email attachments. We used the Enron dataset [32] for collecting the general email attachment size distribution. We ignored files with size less than 500 B. The average size of an attachment was 193.6 kB in the Enron dataset⁷, while the median was 98.8 kB. We separated out the executables (files with extensions .exe, .dll, etc.), which had a mean size of 13.7 kB and a median of 3.8 kB. We conducted our experiments based on such statistics.

For the malware blocklist, we used the Ember dataset [29], embedding the 16,356,790 malware instances to clusters of different radii.

⁶ <https://github.com/zxinyuan/Half-Moon-Cookie>

⁷ <https://trec-legal.umiacs.umd.edu/corpora/trec/legal10/>

6.2 Explicit check

Choosing a hash function Unlike cryptographic hashes, malware hashes (e.g., locality sensitive hashes, context triggered piecewise hashes) support similarity estimation between files. Common choices are TLSH [42], ssdeep [34], and sdhash [45], which are widely used in platforms such as VirusTotal and VirusShare. Their accuracy is typically tuned via a similarity threshold on digests. The computational and communication costs of *Half Moon* are independent of this threshold. Practitioners can therefore choose thresholds to meet accuracy goals without affecting efficiency.

We evaluate the RGC-based optimization from Sec. 5.2 for implementing these hashes within the Embed-and-Map protocol (Π_{EM}), as shown in Table 1. The table reports communication and response time, measured at C_{snd} from invocation to completion against a lightly loaded S. For each hash, we compare implementations without (✗, standard garbled circuits) and with (✓) RGCs. The input size is 5 kB, roughly the median size of an email-attachment executable. Across all three hash functions, our RGC optimization reduces Π_{EM} communication by over two orders of magnitude and response time by at least one order. The no-RGC results in Table 1 are a very conservative lower bound for a monolithic garbled-circuit implementation of explicit check. TLSH yields the fastest response time and ssdeep minimizes communication volume. Π_{EM} using sdhash (with RGCs) incurs roughly 4× the latency of ssdeep and 5× that of TLSH, and about 2× the communication of the others.

Although ssdeep is otherwise efficient, it embeds into edit distance, which is incompatible with *Half Moon*. To use it in explicit check, we further embed edit distance into Hamming distance via edit-sensitive parsing [18], so that the embedding will be compatible with Chakraborti, et al. [13]. This adds 1.23 s latency and 4.46 MB communication in Π_{EM} , and requires larger embedding vectors to offset approximation error, increasing the cost of Π_{TC} . Consequently, ssdeep yields a higher end-to-end explicit check latency than TLSH, as reflected in Table 2a and Table 2b.

We also tested using sdhash, which transforms w to a Bloom filter whose size scales linearly with w . This expansion significantly increases the communication and computation required during $\mathcal{F}_{TC}$, ultimately slowing overall performance. As such, TLSH outperforms both ssdeep and sdhash in the the overall explicit check for all parameter settings we tested (Table 2). Therefore, we pick TLSH for the rest of our *Half Moon* protocol benchmarking.

hash	RGCs	Comm. (MB)	Response Time (s)
TLSH	✗	6.8e4	1.0e3
	✓	6.0e1	4.8e0
ssdeep	✗	5.5e3	2.0e2
	✓	5.3e1	5.4e0
sdhash	✗	5.8e4	9.7e2
	✓	1.3e2	2.4e1

Table 1: Embed-and-map (Π_{EM}) costs with (✓) and without (✗) reusable garbled circuits (RGCs), with w of size 5 kB

size of w (kB)	$ \mathcal{L} $					$ \mathcal{L} $					$ \mathcal{L} $				
	1e2	5e2	1e3	5e3	1e4	1e2	5e2	1e3	5e3	1e4	1e2	5e2	1e3	5e3	1e4
5e0	14	45	83	391	781	45	169	319	1548	3113	46	77	114	422	813
1e1	19	50	88	397	786	62	187	337	1567	3130	101	132	170	477	868
1e2	111	138	178	485	876	338	456	607	1839	3400	658	684	715	1032	1420
2e2	209	240	283	587	978	632	757	905	2137	3710	1254	1282	1325	1631	2026
1e3	1013	1040	1078	1388	1773	3033	3161	3319	4548	6101	6057	6085	6121	6428	6819

(a) TLSH
(b) ssdeep
(c) sdhash

Table 2: Explicit check response time (s) for different fuzzy hashes.

Explicit check throughput To evaluate throughput of explicit check in *Half Moon*, we increased the number of concurrent client requests to S until saturation. Results are shown in Fig. 4, where the x-axis is the number of concurrent requests and the y-axis reports the response time per client request, averaged over all clients’ invocations.

The black solid line in both plots corresponds to C_{snd} with input size 10 kB and $|\mathcal{L}| = 100$. Response time is low and increases gradually at small concurrency levels, with a single request completing in 19.11 s. However, as the concurrency level increases, response time reaches a noticeable inflection point around 250 simultaneous requests. At this load, the average response time spikes, suggesting that the server reached a saturation threshold beyond which queuing and resource contention dominate system performance. The red solid line represents the same functionality in a monolithic garbled circuit. It takes 247 s for a single client to finish the task, while the performance significantly degrades at around five clients.

In Fig. 4a, increasing the blocklist size $|\mathcal{L}|$ leads to faster throughput degradation, as larger sets increase communication cost and network overhead. Consequently, the saturation point shifts to lower concurrency levels: around 80 clients for $|\mathcal{L}| = 1,000$ and about 20 clients for $|\mathcal{L}| = 10,000$.

In Fig. 4b, varying the input size has a much smaller impact. Thanks to reusable garbled circuits (Sec. 5.2), server-side workload is independent of input size, and only client-side computation scales. Thus, latency increases modestly with input size, while the overall throughput curve and saturation point remain largely unchanged.

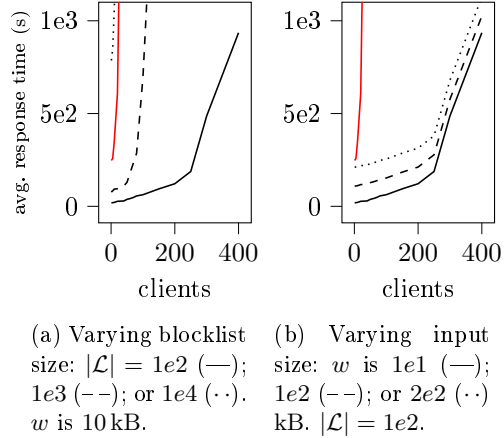


Fig. 4: Explicit check throughput; note that the axes are linearly scaled.

6.3 Implicit check

As discussed in Sec. 7, the design of *Half Moon* is closely related to an asymmetric fuzzy PSI, which enables one party to learn approximate matches between its set and another party’s input under a given distance metric. We compare implicit check to a one-shot fuzzy PSI in Table 3 for two reasons. First, implicit check amortizes matching costs by avoiding repeated proximity checks against the blocklist when the same C_{snd} input is validated multiple times. Second, it mitigates TOCTOU attacks: in a naive fuzzy PSI approach, the proximity check on w is decoupled from its use unless recomputed each time. Thus, asymmetric fuzzy PSI serves as a natural baseline with comparable functionality and security.

We compare with Chakraborti, et al. [13] and Blass and Noubir [10], which are Hamming-distance-based and support asymmetric set sizes. With TLSH embeddings of 35 bytes, Chakraborti, et al.’s outperforms Blass and Naoubir’s, since its cost is independent of vector length. We also include Randal and Schoppmann [44], a state-of-the-art semi-honest exact PSI that operates on $\mathcal{U}_{\text{blk}}$ rather than embeddings $\mathcal{L}$.

As Π_{IC} requires only a few $\mathcal{F}_{ole}$ executions for reconstructing the token and then performs an exact match over the token, it significantly outperforms either a fuzzy PSI or an exact PSI by at least two orders of magnitude in response time and three orders of magnitude in communication volume. Moreover, its cost is independent of the blocklist size.

	Comm. (MB)	Response Time (s)
Π_{IC}	$7.2e-3$	$1.9e-1$
[13]	$3.4e1$	$1.6e2$
[10]	$2.0e3$	$2.6e3$
[44]	$6.3e2$	$6.1e1$

Table 3: Implicit check cost comparison on w of size 100 kB.

7 Related Work

Outsourced malware defense: A driving application for *Half Moon* is to enable a sender of an executable to perform an explicit check with a policy server so that recipients can more efficiently confirm the result via an implicit check, enabling recipients to avoid TOCTOU attacks. Our framework is best suited to malware detection by fuzzy hashing [37, 45] and provides the content privacy necessitated by outsourcing this scanning to a third-party service with a proprietary blocklist. CloudAv [41] was the first work to outsource malware scanning to a cloud that runs the anti-virus engines. SplitScreen [12] and RScam [48] enhance client and malware vendor privacy by sending only compact representations of the files/malware signatures over the network, but provide no cryptographic protections to the data. Perhaps closest to our envisioned design, Sun, et al. [49] proposed a privacy-preserving cloud-based malware detection scheme. However, it exactly matches client input files with malware signatures, necessitating a very large blocklist—and resulting high protocol cost—to achieve effective coverage.

Private blocklist matching: A line of research has focused on enabling a client to query whether an item appears on a blocklist while preserving the

confidentiality of the client’s input. This paradigm has been adapted to applications such as spam filtering [31], and credential breach checks [36]. Kogan and Corrigan-Gibbs [33] provide a generic private blocklist lookup construction with two-server PIR. Private Hash Matching [35] is a service designed for end-to-end encrypted messaging, where a server matches a client’s media content against a set of images (e.g., child sexual abuse material) while hiding the server’s hash set or nonmatching content from the client. But none of the above schemes generate a hiding and binding token after the match to allow for faster verifications later on the same content (as in our implicit check).

Some OPRF variations are similar to our *Half Moon* primitive, as well. Programmable OPRFs (OPPRF) [14] and oblivious key-value stores (OKVS) [22] both “program” an OPRF so that the server gets a PRF output for an input w iff w matches a server list item, while the output is pseudorandom elsewhere. All solutions above support only exact matching against the blocklist, requiring everything to be blocked to be explicitly enumerated on the blocklist. Scannapieco, et al. [46] consider approximate matching over database records but rely on a trusted third party to facilitate the check.

PSI-based mechanisms: A part of our design relies on approximately matching an item against a set privately (e.g., using fuzzy PSI mechanisms). Structure-aware PSIs [23,24] rely on spatial hashing, which guarantees reasonable compute costs only under certain assumptions about the data in one (or both) of the input sets. Approximate PSI [16] is a similarly motivated concept that also works only under certain distribution assumptions. As it is unclear if these distribution assumptions hold across a variety of applications, we cannot generally use a structure-aware PSI or an approximate PSI as a building block for a *Half Moon*.

Distance-aware PSI [13] and *Fuzzy* PSI [10] perform fuzzy matching over two sets *without any data distribution assumptions*. While these incur higher costs than both structure-aware PSI and approximate PSI, distance-aware PSI and fuzzy PSI can be applied more broadly because they are agnostic to the data distribution. We use Chakraborti, et al. [13] as a building block for Π_{TC} , as we detail in the full version [54]. Unlike Chakraborti, et al. [13], the method of Blass and Noubir [10] operates directly on binary vector inputs and is therefore not compatible with our Π_{EM} . All above solutions either assume a semi-honest client or require non-trivial modifications to ensure a consistent client input between the fuzzy PSI matching and the computation of a hiding and binding token to store at S , to enable a subsequent implicit check. *Half Moon* does not trust the client during an explicit check and, in particular, detects when the client submits inconsistent inputs from embedding to subsequent checking.

Beyond application-agnostic designs, Wang, et al. [51] realize an approximate edit-distance check on genome data. Similar to a *Half Moon*, they utilize an embedding phase and a test phase to compare entries at the client and the server. Also related is similarity-aware compromised credential checking (C3) [43]. These solutions work in semi-honest models only, however. In addition, *Half Moon* removes the assumptions on the structure of input data in Wang, et al. [51], which is tailored to genome data and may not be applicable to other data.

Protocol	Computation	Communication	Client	TOCTOU resilient
<i>Half Moon</i>				
explicit check	$\text{GC}_f(\ell) + \text{FPSI}(\mathcal{L})$	$\text{GC}_f(\ell) + \text{FPSI}(\mathcal{L})$	Mal	✓
implicit check	$\text{EMB}(\ell) + \text{OLE}$	OLE	SH	✓
PSI	$\text{PSI}(\mathcal{U}_{\text{blk}})$	$\text{PSI}(\mathcal{U}_{\text{blk}})$	Mal	✓
Fuzzy PSI	$\text{EMB}(\ell) + \text{FPSI}(\mathcal{L})$	$\text{FPSI}(\mathcal{L})$	SH	✓
Garbled Circuit	$\text{GC}_f(\ell) + \text{GC}_{\text{blocked}}(\mathcal{L})$	$\text{GC}_f(\ell) + \text{GC}_{\text{blocked}}(\mathcal{L})$	Mal	✓
Blind Signature	$\text{PSI}(\mathcal{U}_{\text{blk}})$	$\text{PSI}(\mathcal{U}_{\text{blk}})$	Mal	✗

Table 4: Protocol comparison. GC_f and $\text{GC}_{\text{blocked}}$ denote the costs of executing a garbled circuit computing f or blocked , respectively. PSI and FPSI denote the costs of plain PSI and fuzzy PSI, respectively. OLE denotes the cost of performing an OLE, and EMB denotes the cost of running the embedding algorithm f outside the garbled circuit. ℓ denotes the size of the client input w . “Mal” and “SH” indicate security against malicious and only semi-honest clients, respectively.

In Table 4, we summarize the costs and key guarantees of *Half Moon* and several cryptographic alternatives.

8 Conclusion

This paper presents *Half-Moon Cookie*, which enables a client to determine if its item approximately matches any item of a server’s blocklist, without requiring the client to disclose its item to the server or the server to disclose its blocklist to the client. If the client’s item passes this check, the server is provided a hiding and binding token to that item to store on its allowlist until the blocklist is updated, so that future clients can more efficiently confirm that the item they received was checked against the latest blocklist. We apply *Half Moon* to malware defense, wherein a sending client performs the (relatively expensive) blocklist check before sending an executable, permitting receiving clients to perform only the (relatively inexpensive) allowlist check before using the executable, enabling them to efficiently avoid TOCTOU attacks. Through judicious protocol construction, *Half Moon* scales efficiently in the sizes of both the item to be checked and the blocklist.

Acknowledgments This work was supported in part by NSF grant 2207214.

References

1. Don’t route or peer lists (DROP). <https://www.spamhaus.org/blocklists/do-not-route-or-peer/>
2. emp-toolkit. <https://github.com/emp-toolkit>
3. NTL: A library for doing number theory. <https://libntl.org>
4. CWE-367: Time-of-check time-of-use (TOCTOU) race condition. <https://cwe.mitre.org/data/definitions/367.html> (2006)

5. Internet x.509 public key infrastructure certificate and certificate revocation list (CRL) profile. <https://datatracker.ietf.org/doc/html/rfc5280> (2008)
6. EasyList. <https://easylist.to> (2016)
7. Customer advisory: An attack, leveraging spam email, Teams, SharePoint, and OneDrive, potentially linked to the ransomware group Sangria Tempest. <https://www.deepwatch.com/labs/spam-email-attach-teams-sharepoint-and-onedrive-potentially-linked-to-the-ransomware-group-sangria-tempest/> (2024)
8. Black and white cookie. https://en.wikipedia.org/wiki/Black_and_white_cookie (8 Sep 2025)
9. Belotti, J.: More than DNS: The 14 hour AWS us-east-1 outage. <https://thundergolfer.com/blog/aws-us-east-1-outage-oct20> (26 Oct 2025)
10. Blass, E., Noubir, G.: Assumption-free fuzzy PSI via predicate encryption. <https://eprint.iacr.org/2025/217> (2025)
11. Broadcom: About the SEP Intensive Protection file reputation service. <https://techdocs.broadcom.com/us/en/symantec-security-software/information-security/data-loss-prevention/16-1/about-discovering-and-preventing-data-loss-on-endpoints/about-the-sep-intensive-protection-file-reputation-service.html> (4 Sep 2025)
12. Cha, S.K., et al.: SplitScreen: Enabling efficient, distributed malware detection. In: 7th USENIX NSDI (Apr 2010)
13. Chakraborti, A., et al.: Distance-aware private set intersection. In: USENIX Security (2023)
14. Chandran, N., et al.: Circuit-PSI with linear complexity via relaxed batch OPRF. In: PoPETS (2022)
15. Charikar, M.S.: Similarity estimation techniques from rounding algorithms. In: ACM STOC (May 2002)
16. Chongchitmate, W., et al.: Approximate PSI with near-linear communication. <https://eprint.iacr.org/2024/682> (2024)
17. Cisco: Filereputation filtering and file analysis. https://www.cisco.com/c/en/us/td/docs/security/ces/ces_15-5-3/user_guide/b_ESA_Admin_Guide_ces_15-5-3/b_ESA_Admin_Guide_12_1_chapter_010001.pdf
18. Cormode, G., Muthukrishnan, S.: The string edit distance matching problem with moves. In: ACM SODA (2002)
19. Child sexual abuse material (CSAM) identification and reporting for U.S. based companies. https://technologycoalition.org/wp-content/uploads/CSAM-Identification_Reporting_R3-1.pdf (2022)
20. Datar, M., et al.: Locality-sensitive hashing scheme based on p-stable distributions. In: ACM Symp. Comp. Geo. (2004)
21. DNS blacklists and whitelists. <https://datatracker.ietf.org/doc/html/rfc5782> (2010)
22. Garimella, G., et al.: Oblivious key-value stores and amplification for private set intersection. In: CRYPTO. p. 395–425 (2021)
23. Garimella, G., et al.: Structure-aware private set intersection, with applications to fuzzy matching. In: CRYPTO (2022)
24. Garimella, G., et al.: Computation efficient structure-aware PSI from incremental function secret sharing. In: CRYPTO (2024)
25. Ghosh, S., Nilges, T.: An algebraic approach to maliciously secure private set intersection. In: EUROCRYPT (2019)
26. Ghosh, S., Simkin, M.: The communication complexity of threshold private set intersection. In: CRYPTO (2019)

27. Ghosh, S., et al.: Maliciously secure oblivious linear function evaluation with constant overhead. In: ASIACRYPT (2017)
28. Harth-Kitzerow, C., et al.: CRGC – a practical framework for constructing reusable garbled circuits. In: SECURE (2022)
29. Joyce, R.J., et al.: Ember2024 - a benchmark dataset for holistic evaluation of malware classifiers. In: ACM SIGKDD (2025)
30. Katz, J., et al.: Efficiently enforcing input validity in secure two-party computation. <https://eprint.iacr.org/2016/184> (2016)
31. Kim, I., et al.: PCSF: Privacy-preserving content-based spam filter. IEEE TIFS (Mar 2023)
32. Klimt, B., Yang, Y.: The enron corpus: A new dataset for email classification research. In: ECML (2004)
33. Kogan, D., Corrigan-Gibbs, H.: Private blocklist lookups with checklist. In: USENIX Security (2021)
34. Kornblum, J.: Identifying almost identical files using context triggered piecewise hashing. Dig. Inv. (Sep 2006)
35. Kulshrestha, A., Mayer, J.R.: Identifying harmful media in end-to-end encrypted communication: Efficient private membership computation. In: USENIX Security (2021)
36. Li, L., et al.: Protocols for checking compromised credentials. In: ACM CCS (2019)
37. Li, Y., et al.: Experimental study of fuzzy hashing in malware clustering analysis. In: USENIX Conf. Cyber Sec. Exp. & Test (2015)
38. MalwareBazaar. <https://bazaar.abuse.ch>
39. Microsoft Defender: Turn on cloud protection in Microsoft Defender Antivirus. <https://learn.microsoft.com/en-us/defender-endpoint/enable-cloud-protection-microsoft-defender-antivirus> (20 Oct 2025)
40. Microsoft Threat Intelligence: File hosting services misused for identity phishing. <https://www.microsoft.com/en-us/security/blog/2024/10/08/file-hosting-services-misused-for-identity-phishing> (2024)
41. Oberheide, J., et al.: CloudAV: N-version antivirus in the network cloud. In: 17th USENIX Security (Jul 2008)
42. Oliver, J., et al.: TLSH – a locality sensitive hash. In: Cybercrime & Trust. Comp. Work. (2013)
43. Pal, B., et al.: Might I get pwned: A second generation compromised credential checking service. In: USENIX Security (2022)
44. Rindal, P., Schoppmann, P.: VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE. In: EUROCRYPT (2021)
45. Roussev, V.: Data fingerprinting with similarity digests. In: Adv. Dig. For. (2010)
46. Scannapieco, M., et al.: Privacy preserving schema and data matching. In: ACM SIGMOD (2007)
47. Domain blocklist (DBL). <https://www.spamhaus.org/blocklists/domain-blocklist/>
48. Sun, H., et al.: RScam: Cloud-based anti-malware via reversible sketch. In: 11th SECURECOMM (Oct 2015)
49. Sun, H., et al.: PriMal: Cloud-based privacy-preserving malware detection. In: Australasian Conf. Inf. Sec. & Priv. (2017)
50. URLhaus. <https://urlhaus.abuse.ch>
51. Wang, X.S., et al.: Efficient genome-wide, privacy-preserving similar patient query based on private edit distance. In: ACM CCS. pp. 492–503 (2015)
52. Yao, A.C.: Protocols for secure computations. In: IEEE FOCS (1982)

- 53. Yao, A.C.: How to generate and exchange secrets. In: IEEE FOCS (1986)
- 54. Zhang, X., Chakraborti, A., Reiter, M.K.: Half-moon cookie: Private, similarity-based blocklisting with TOCTOU-attack resilience (2026), <https://arxiv.org/abs/2604.15641>

A One-time pairwise unpredictable function

We will require a function that is *one-time pairwise unpredictable*. Intuitively, a keyed-function $g : \mathcal{K} \times \mathcal{D} \rightarrow \mathcal{R}$ is one-time pairwise unpredictable if given the output at a specific point under a randomly chosen key, an adversary cannot guess the output at another distinct point. The function is one-time in the sense that each new invocation of the function requires a new random key selection to preserve unpredictable outputs. We extend this notion to a *one-time pairwise unpredictable permutation* over vectors in a finite field $\mathbb{F}^\theta$. We define the properties required from such a permutation below.

Definition 4 Let $\mathbb{F}$ be a finite field and $\theta \geq 1$. A keyed-function $g : \mathcal{K}^\theta \times \mathbb{F}^\theta \rightarrow \mathbb{F}^\theta$ is one-time pairwise unpredictable over $\mathbb{F}^\theta$ if for any pair of input $\vec{w}, \vec{w}' \in \mathbb{F}^\theta$ for which $|\vec{w} \Delta \vec{w}'| = d$ and outputs $\vec{x}, \vec{y} \in \mathbb{F}^\theta$,

$$\mathbb{P}(g_k(\vec{w}) = \vec{x}) = \frac{1}{|\mathbb{F}|^\theta} \quad (2)$$

$$\mathbb{P}(g_k(\vec{w}) = \vec{x} \mid g_k(\vec{w}') = \vec{y}) = \left(\frac{|\mathbb{F}|}{|\mathcal{K}|} \right)^d \quad (3)$$

where the probability is over the choice of the key $k \in \mathcal{K}^\theta$. Here $|\vec{w} \Delta \vec{w}'| = d$ means that $\vec{w}$ and $\vec{w}'$ differ in at least d components.